

PYTHON MODULES & PACKAGES

DAY 9 • QUICK & EASY REVISION NOTES

Learn the concept first → remember the syntax → practice the example.

1. Module

Module = a Python file (.py) containing reusable code. It can contain functions, variables, classes and statements.

Why use modules? **Reuse code • Organize code • Easy maintenance • Avoid repetition**

```
calculator.py
def add(a, b):
    return a + b
```

Here **calculator.py** is a module.

2. Importing a Module

```
import math
print(math.sqrt(25))
```

Other common forms:

```
from math import sqrt
print(sqrt(25))

import math as m
print(m.sqrt(25))
```

Remember: import module → use module.function().

3. Custom Module

A custom module is created by us for our own project.

```
mycalculator.py
-----
def add(a, b):
    return a + b

def sub(a, b):
    return a - b

mymain.py
-----
import mycalculator

print(mycalculator.add(10, 20))
print(mycalculator.sub(20, 10))
```

Output: **30** and **10**.

4. Package

Package = a folder containing related Python modules.

```
mypackage/
■■■■ __init__.py
■■■■ calculator.py
■■■■ student.py
■■■■ message.py
```

Simple memory trick: **Module = one file | Package = group of modules.**

5. Python Standard Library

Python provides many ready-to-use modules with Python itself. These are called the **Python Standard Library**.

Module	Main use	Remember
math	Mathematics	sqrt, pow, factorial, ceil, floor, pi
random	Random values/items	choice, randint, randrange, shuffle, sample
datetime	Date & time	now, today, timedelta, strftime

Module	Main use	Remember
calendar	Calendar	month, calendar, isleap
os	Files/folders/OS	getcwd, listdir, mkdir, path.exists
sys	Python/system info	version, platform, argv, path
json	JSON data	loads, dumps, load, dump
re	Text patterns	search, match, findall, sub, split
statistics	Statistics	mean, median, mode, stdev
collections	Special containers	Counter, deque, defaultdict
time	Timing	sleep, time, ctime, perf_counter

6. math Module — Must Know

Used for mathematical calculations.

```
import math

math.sqrt(25) # 5.0
math.pow(2, 3) # 8.0
math.factorial(5) # 120
math.ceil(4.2) # 5
math.floor(4.8) # 4
math.pi # 3.14159...
```

Easy memory: sqrt = root | pow = power | factorial = n! | ceil = up | floor = down | pi = π

Note: **math.add()**, **math.max()**, **math.min()** are not math functions. max(), min(), sum() are built-in functions.

7. random Module — Must Know

```
import random

names = ["Arun", "Rahul", "Amit", "Rohit", "Vivek"]

print(random.choice(names))
print(random.randint(1, 100))
```

choice() → one random item

randint(a,b) → random integer, both ends included

randrange(1,10) → 1 to 9

shuffle(list) → changes list order

sample(list,2) → 2 unique random items

random() → 0 to less than 1 float

uniform(10,20) → random float in range

```
random.shuffle(names)
print(names)

print(random.sample(names, 2))
```

Most important: choice, randint, shuffle, sample.

8. datetime Module

```
import datetime

print(datetime.datetime.now())
print(datetime.date.today())
```

Useful: **now()** = current date & time | **today()** = current date | **timedelta()** = date/time difference | **strftime()** = format date as text.

9. calendar Module

```
import calendar

print(calendar.month(2026, 9))
print(calendar.isleap(2024))
```

Useful: **month()**, **calendar()**, **isleap()**, **monthrange()**.

10. os Module

```
import os

print(os.getcwd())
print(os.listdir())
print(os.path.exists("pythonModule"))
```

Useful for files, folders, directories and operating-system operations.

11. sys Module

```
import sys

print(sys.version)
print(sys.platform)
print(sys.argv)
```

Used for Python interpreter and system-related information.

12. json Module

```
import json

data = '{"name":"Arun", "age":23}'
obj = json.loads(data)
print(obj["name"])

text = json.dumps(obj)
print(text)
```

loads() → JSON string to Python object

dumps() → Python object to JSON string

load()/dump() → work with JSON files.

13. re Module

```
import re

text = "My marks are 85 and 92"
print(re.findall(r"\d+", text))
```

Used for pattern matching, searching, validation and text replacement.

Common: **search()**, **match()**, **findall()**, **sub()**, **split()**.

14. statistics Module

```
import statistics

marks = [70, 80, 90, 60, 100]
print(statistics.mean(marks))
print(statistics.median(marks))
```

Important: **mean()** = average | **median()** = middle value | **mode()** = most frequent | **stdev()** = standard deviation.

15. collections Module

```
from collections import Counter

names = ["Arun", "Amit", "Arun", "Rahul", "Amit", "Arun"]
print(Counter(names))
```

Important: **Counter** counts frequency | **deque** supports fast operations at both ends | **defaultdict** provides default values.

16. time Module

```
import time

print("Hello")
time.sleep(2)
print("Python")
```

sleep() pauses execution. **time()** gives a timestamp. **perf_counter()** is useful for measuring elapsed time.

17. Final One-Page Revision

MODULE → One .py file
PACKAGE → Collection of related modules
IMPORT → Use another module

math → calculations
random → random values
datetime → date/time
calendar → calendar
os → files/folders/OS
sys → system/interpreter
json → JSON data
re → patterns
statistics → statistics
collections → special containers
time → timing

Day 9 Golden Rule: Don't try to memorize every method. First understand what each module is used for, then remember its 3–5 most common methods.

GitHub naming suggestion: **Day_9_Python_Modules_and_Packages_Quick_Notes.pdf** is clear, professional, and easy to search.